智控星海 · StarSea

手势驱动的天文科普可视化教学工具

技 术 文 档

—— 全国高校 Vibe Coding 开发挑战赛 · 省赛参赛作品 ——

作品名称	智控星海（StarSea）
赛　道	Vibe Coding 开发挑战赛（具体报名组别待核实）
技术栈	TypeScript · Three.js · WebGL2 · MediaPipe Tasks-Vision · Vite
代码规模	TypeScript / GLSL 源码（行数与文件数待按最终仓库统计）
文档版本	v1.0

本文档所有性能数据以《实测记录表》为准，未测量的指标不作估计。

目 录

1. **项目概述** 作品定位、目标用户、交付形态

2. **问题定义与选题依据** 痛点、必要性论证、赛道方向对应

3. **产品设计** 功能闭环、用户旅程、教学映射

4. **系统架构设计** 分层、数据流、模块职责、关键决策

5. **手势意图控制设计** 关键点→连续量、滤波、鲁棒性、双模式

6. **GPGPU 粒子模拟管线** 双纹理、速度场、渲染、质量分级

7. **六个场景的实现** 逐场景技术要点与对照

8. **AI 与 Skills 协同开发** **工具分工、上下文筛选、迭代复盘**

9. **工程质量** 类型安全、资源生命周期、缺陷复盘

10. **可复现性论证** 环境、步骤、验证清单、证据链

11. **开源合规与原创性** 上游披露、改动清单、许可义务

12. **局限与后续规划** 已知不足、演进路线

A. **附录：关键配置参数** 六场景核心常量

B. **附录：快捷键与交互索引**

1 项目概述

1.1 一句话说明

「智控星海」是一个运行在浏览器里的实时粒子可视化工具：使用者站在摄像头前，用手掌的开合、移动与双手间距，直接操控六个天体场景的形态演化，把一个抽象的物理量变成看得见、动得了的画面。

> 产品定位：不是粒子特效演示，而是天文与物理课堂上的「可交互教具」。判断标准很简单——如果一段演示不能对应到一个具体的知识点，它就不进这个作品。

1.2 目标用户与使用场景

三类人：中学/大学物理与地理教师（课堂投影演示，诉求是不用回讲台、站在投影前就能讲，对应「摄像头手势优先 + 鼠标完整兜底」）；科技馆与校园开放日的讲解员（展台互动，诉求是观众不解释也会玩，对应「启动页二选一 + HUD 实时提示手势状态」）；以及课后自学的学生（诉求是能复现老师演示的每一个状态，对应「场景快捷键 + 参数面板 + 一键复位」）。

1.3 交付形态与运行环境

纯前端单页应用，无后端、无数据库、无服务端接口。摄像头视频帧只在浏览器本地参与计算，不上传任何数据。构建产物是一组静态文件，可部署到任意静态托管。

项	规格
运行入口	浏览器打开静态页面（开发：Vite dev server；生产：dist/ 静态产物）
渲染后端	WebGL2（必需）；不支持时给出明确降级提示而非白屏
计算位置	100% 客户端。手部推理在本地 WASM/WebGPU 上执行，视频帧不离开浏览器
网络依赖	MediaPipe 模型与 WASM 从公开 CDN 加载；浏览器缓存不保证后续离线可用
构建校验	tsc --noEmit 强制零错误后才允许打包

2 问题定义与选题依据

2.1 被解决的问题

天文与物理教学里有一类内容天然难以呈现：它们不是「一个东西长什么样」，而是「一个参数变了，形态怎么跟着变」。这类内容有三个特征——过程慢（恒星演化）、尺度大（星系结构）、不可见（磁场、引力场）。

现有手段各有硬伤：

- 静态图片：只能定格一个瞬间，学生看不出「从 A 到 B」的中间过程。
- 视频：能展示过程，但单向播放，学生无法改变任何变量，本质仍是「看」而不是「做」。
- 仿真软件：能改参数，但交互依赖复杂的参数面板，课堂节奏会被打断。

本项目选择的切入点是：把「改变变量」这个动作，从操作面板转移到身体本身。

2.2 为什么是手势，而不是鼠标

这是本项目最重要的取舍。手势不是「更酷的鼠标」，它解决的是一条具体约束：讲课时人的位置不能动。老师站在投影前、面向学生，手已经在镜头里，动作即输入；而鼠标要求他回到讲台、找光标、再定位，课堂节奏就断了。

- 手势的代价是稳定性（受光照与遮挡影响），补偿方式是「手势优先、鼠标提供基础操作兜底」：两种输入产出同一个 GestureFrame，下游逻辑无感知，两种输入的能力并非完全等同。

- 手势的可发现性低，补偿方式是启动页给出模式选择与手势说明，进入后再用 Toast 复述要点。

2.3 与赛道推荐方向的对应

赛题列出 10 个推荐方向，本项目落在其中两个的交集上。方向 5「AI 驱动的高交互教育与可视化」要求把抽象概念转化为可互动的可视化，本项目对应六个场景分别讲解引力收束、旋臂差速、行星环结构、极光成因、生理节律与抛体运动，且交互不止滑块，还包括手势。方向 10「智能游戏原型与交互式叙事工坊」强调快速搭建可交互原型，本项目六个场景共享同一引擎、可即时切换的结构本身就是「场景插件化」的验证——心跳与礼花两个非天文题材证明这套引擎不局限于天文，是一个可复用的粒子场底座。

3 产品设计

3.1 功能闭环

一个教学工具要「能用」，必须走完「进入 → 演示 → 解释 → 退出」四步。进入：启动页说明手势，权限申请与模型加载状态可见（StartScreen / WebcamPanel）；演示：挥手、握拳、双手开合，形态连续变化，HUD 同步显示当前手势量（HandTracker → InputManager → StateMachine）；解释：按 G 切场景、按 S 调参数，Toast 提示场景名与知识点关键词（Overlay / ParamPanel）；退出：按 Esc 断开摄像头并回到启动页；粒子渲染资源未在此动作中全部释放。

3.2 用户旅程中的关键设计

启动页不是一句话「请允许摄像头」，而是并排给出两条路径与各自的手势图示，并在选中后立即用 Toast 复述该模式下的操作要点，首次使用无需读文档即可上手。

运行期有两条保底设计。其一是「状态永远可见」：HUD 常驻显示 FPS、输入模式（hand / hand·lost / mouse / collide）、场景名、当前调节量名称与左右手是否在线——看不见的系统状态是最容易让人放弃的地方。其二是「失败要优雅」：手势丢失不是直接归零，而是分四级降级——0–60ms 不透明度保持 1；60–300ms 线性淡出、形态量同步衰减；300–1000ms 全部归位到中性并显示 hand·lost；超过 1000ms 判定为无手，自动落回鼠标帧，画面始终可操作。

3.3 六个场景的教学映射

场景	手势 0 → 1	对应知识点	课堂上可以问的问题
星云 nebula	稀疏湍流云 → 致密发光球	分子云、引力收束	为什么云会变成球？是什么力在起作用？
星系 galaxy	平静旋臂 → 星锻熔铸	旋臂密度波、差速旋转	为什么旋臂不会越缠越紧？
地球 ocean	自转地球 → 极光风暴	自转、极光成因	极光为什么只在两极出现？
土星 saturn	平静环系 → 环面坍缩	行星环、卡西尼缝	环为什么有缝？内环为什么转得快？
心跳 heart	62 BPM → 128 BPM 暗核	心肌收缩、搏动波	两次「咚」之间发生了什么？
礼花 pulse	弹道礼花 → 离子环	抛体运动、冲击波	为什么炸开的火花会落回来？

4 系统架构设计

4.1 总体分层

系统分为三层，层间只通过明确的数据结构通信，不共享可变状态：

层	核心组件	输入	输出

层	核心组件	输入	输出
输入层	HandTracker（MediaPipe 21 关键点，30Hz，GPU delegate） MouseInput（轨迹球 + 左键握拳 + 滚轮锁存） InputManager（手/鼠标选择、双模式、双手中点与引力轴）	摄像头帧、鼠标与键盘事件	GestureFrame
状态层	StateMachine（场景索引、uniform 目标值、逐帧平滑、颜色插值、事件阈值、力场球驱动）	GestureFrame + 当前场景	约 90 个 uniform 目标值
模拟与渲染层	ParticleSystem（GPUComputationRenderer 双纹理 ping-pong） THREE.Points（1 顶点/纹素）→ UnrealBloom → OutputPass	uniform + 纹理	最终帧缓冲

数据自下而上单向流动：输入层每帧产出一个 GestureFrame，状态层把它翻译成 uniform，模拟层只认 uniform、不知道场景语义。反向没有通道——渲染层从不回读手势状态，这也是 GPGPU 能保持零 CPU 回读的前提。

4.2 每帧数据流

主循环固定为七个阶段，顺序不可交换（后一阶段依赖前一阶段的输出）：

1. input.poll() —— 取回 GestureFrame（含 dial、旋转增量、力场球、能量、双手中点）
2. stateMachine.update() —— 把 GestureFrame 翻译成约 90 个 uniform 的目标值并平滑
3. 力场球位置世界坐标 → 粒子局部坐标（worldToLocal），消除旋转对碰撞的影响
4. particles.update() —— GPU 两趟计算（速度 → 位置），结果写回纹理
5. 轨迹球惯性积分 —— 角速度衰减、上限钳制、自动慢旋回落
6. centerStar.update() —— 星系核心恒星/黑洞网格状态
7. overlay.update() + composer.render() —— HUD 刷新与后期合成

> 第 3 步容易被忽略：粒子场本身在旋转（轨迹球），若直接把世界坐标传给着色器做碰撞，旋转后手和粒子的相对位置会错位。这里每帧把手的坐标转换到粒子局部系，使「手推开粒子」在任意旋转角度下都成立。

4.3 模块职责

模块	行数	单一职责	不允许做的事
main.ts	370	装配与主循环、事件绑定	不写物理逻辑
config.ts	213	全部可调常量的唯一来源	不含任何运行时状态
HandTracker.ts	303	摄像头/模型生命周期、关键点→跟踪手	不决定手势语义
InputManager.ts	255	多源输入归一为 GestureFrame	不碰渲染
StateMachine.ts	360	手势量 → uniform 目标值 + 平滑 + 事件	不直接调用 GPU
ParticleSystem.ts	364	GPGPU 资源、播种、材质	不知道场景语义
velocity.frag.glsl	548	六个场景的力场	不含颜色逻辑
points.frag.glsl	259	六场景着色、星芒、极光	不含力
ui/（Overlay / StartScreen / ParamPanel / WebcamPanel）	901	HUD、设置、帮助、Toast、首屏、参数编辑、摄像头预览	不读写 uniform，不绕过 config

4.4 四个关键设计决策

决策一：一切常量外置到 config.ts

213 行、约 200 个常量，覆盖物理、颜色、手势阈值与性能阈值。三条理由：调参不必改着色器，反馈周期从分钟级降到秒级；AI 天然喜欢把魔数写进逻辑里，而「常量必须外置」这条硬约束能把它的输出强制变成可被人接管的形态；评审可以只读一个文件就理解全部设计意图。

决策二：模拟放在 GPU，而不是 CPU

100 万粒子下，CPU + 类型化数组每帧百万次积分必然掉帧；CPU + Web Worker 虽能并行，但每帧要传输 100 万 × 6 个 float，带宽瓶颈更严重。只剩 GPU 双纹理 ping-pong：一次全屏 pass 完成全部粒子，

带宽留在显存内部。代价是逻辑从 TypeScript 搬到 GLSL，调试困难、不能断点；这个代价通过「所有场景参数走 uniform」与「单场景独立函数」两个约定抵消，出问题时可以把一个场景的力系数归零做隔离，不必整体回滚。

决策三：场景是插件，不是分支堆

新增场景只需四处改动：SCENES 数组、config 中的场景常量、velocity 中的一个 *Force() 函数、StateMachine 中的一个 case；着色按 uScene 分发。这个约束让第六个场景的接入成本低于第一个场景的三分之一。

决策四：双输入在架构层同构

鼠标不是「降级方案」，而是与手势平级的数据源。两者都产出 GestureFrame，InputManager 只在「有手势帧就用手势帧，否则用鼠标帧」一处判断。结果是任何手势能做的高级交互鼠标都能做，任何手势场景的 bug 都能用鼠标复现。

5 手势意图控制设计

5.1 从 21 个关键点到 5 个控制量

MediaPipe HandLandmarker 每帧返回每只手 21 个归一化关键点（x, y ∈ [0,1]，含深度分量）。这些点本身噪声大、维度高，不能直接驱动动画。管线做了三次降维：

```
21 关键点/手
   ↓ ① 空间归一化：所有距离量除以掌宽（腕点→中指 MCP 距离）
      — 消除手大手小、离镜头远近带来的尺度差异
   ↓ ② 几何量提取：closedness / spread / pinch / indexExtended
   ↓ ③ 时间平滑：一欧元滤波（位置）+ 一阶低通（姿态量）
5 个控制量 + 1 个世界坐标
```

掌宽归一化是这一步的关键。没有它，同一个人在 40cm 和 80cm 处做同一个握拳动作，算出的 closedness 完全不同——手势灵敏度会随距离剧烈漂移。

5.2 一欧元滤波：在抖动与延迟之间取平衡

手部关键点存在两个矛盾的需求：静止时要稳（否则光标发抖），快速移动时要跟手（否则有明显延迟）。固定截止频率的低通滤波只能二选一。

本项目采用 One Euro Filter（Casiez 等，CHI 2012），其截止频率随速度自适应：

```
fc = minCutoff + beta * |dx/dt|

静止时 |dx/dt| → 0，fc → minCutoff（1.0 Hz）：强平滑，抖动被压掉
快速移动时 |dx/dt| 增大，fc 升高：平滑减弱，延迟被压掉
```

参数	取值	作用
minCutoff	1.0	静止时的截止频率。越小越稳，但起手会「拖」
beta	0.02	速度对截止频率的增益。越大越跟手，但伴随抖动
dCutoff	1.0	对速度本身再做一次滤波，避免速度噪声反噬

三个通道（x / y / depth）各自独立滤波，因此手在深度方向快速前后移动时不会被横向平滑拖累。参数可在运行时通过设置面板调整，改完立即作用于下一帧。

5.3 四个鲁棒性机制

机制	要解决的问题	做法
幻影剔除	双手靠近时同一只手被检成两只	以掌心 5 点质心为准，两只手质心距离小于 0.09（归一化坐标）时丢弃置信度较低的那只

机制	要解决的问题	做法
按标签绑定	检测顺序逐帧跳变，按索引绑定会让左右手瞬间互换、画面猛跳	按 handedness 标签绑定跟踪对象；同一标签出现多次时取置信度最高者
镜像修正	自拍画面被 MediaPipe 按未镜像标注，抬左手却驱动右手	mirrorHandedness 开关在取值时翻转标签，并做成配置项以适配外接摄像头
深度基线自适应	单目摄像头无法直接得到深度	用掌宽相对基线的变化推断前后位移；基线以 0.5% 速率缓慢跟随，既能适应不同人的手，也不会被一次握拳带偏

5.4 两种交互模式的取舍

模式	手的作用	适合演示	不适合
dials（默认）	左手开合 = 调节盘；右手位移 = 旋转	讲「一个变量的连续影响」，如星云收束、土星环坍缩	讲「多个物体互相作用」
collider	双手本身作为带速度的排斥球	讲「力场」「碰撞」「扰动传播」	精确控制形态

两模式用 C 键切换，切换时清空历史状态（prevRight、力场球初始化标记），避免跨模式的状态污染。轨迹球旋转只在 dials 模式生效——在 collider 模式下，手的位移已经用于推动粒子，再叠加旋转会让两种意图互相干扰。

6 GPGPU 粒子模拟管线

6.1 双纹理 ping-pong

每个粒子在两张浮点纹理中各占一个纹素：位置纹理存 xyz，速度纹理存 xyz。两张纹理互为输入输出，每帧交替读写：

```
步骤 1   读 [pos_tex] + [vel_tex]        --->  写 [vel_tex']   速度 pass
步骤 2   读 [vel_tex'] + [pos_tex]       --->  写 [pos_tex']   位置 pass
步骤 3   THREE.Points 采样 [pos_tex'] 得位置、[vel_tex'] 得速度 -> 颜色

两张纹理互为对方的输入，每帧交替读写，故称 ping-pong。
纹理布局：size x size 个纹素，一个纹素 = 一个粒子。
```

渲染时不需要把位置读回 CPU：顶点着色器的三个分量全部来自纹理采样，几何体本身只有一个 ref 属性（每个顶点对应哪个纹素）。这是能够撑起 100 万粒子的根本原因。

> 纹素坐标偏移 0.5 是必须的：纹理采样以纹素中心为坐标，直接用 i/size 会取到相邻纹素的插值边界。代码中 ref 属性计算为 ((i % size) + 0.5) / size。

6.2 速度场：六个场景的力

velocity.frag.glsl 共 548 行，主体是六个互不耦合的 *Force() 函数，由 main() 按 uScene 分发。每个粒子的速度由三部分叠加：

```
v' = (v + F_scene * dt) * damping      // damping = 0.965
     + F_light  * dt                    // 光/能量扰动
     + F_collider * dt                  // 手部力场球（可选）
v' = clamp(v', 0, maxSpeed)             // maxSpeed = 4.2

F_scene 内部统一的四项：
  F_spring    = (target - p) * stiffness        // 形态回位
  F_curl      = curlNoise(p * freq + t) * turb  // 湍流
  F_sceneSpecific                                // 各场景特有（轨道/重力/冲击波…）
  F_damp      = -v * extraDamping               // 场景所需的额外阻尼
```

> 位置积分本身只有 23 行（p' = p + v·dt），讲究的是 dt 的钳制：标签页切回前台时 clock.getDelta() 可能返回数秒，直接积分会让粒子瞬间飞出场景。因此 maxDelta 固定为 1/30 秒，并在 visibilitychange 时主动丢弃一次 delta。

6.3 渲染与后期

阶段	做法	目的
点精灵	顶点着色器按深度衰减点尺寸；片元着色器按速度在三色渐变上取色	让运动快的粒子更亮，形成「能量感」
形状	星云/星系用环状 sprite，星锻用圆点，星芒用十字	同一引擎下区分「颗粒」与「火花」两种质感
混合	Additive blending（加性）+ depthWrite = false	粒子叠加自然发光；深度写入会互相遮挡出现黑边
密度归一	uDensityNorm = 65536 / count	三档画质（65k/262k/1M）视觉总亮度一致
后期	UnrealBloom(strength 0.65, radius 0.45, threshold 0.72) + OutputPass	发光与色调映射；心跳场景强度随 beat 脉冲

> 双手控制模式另叠一层与场景无关的深度雾：远处粒子压到 0.70x、近处提到 1.12x。该效果由共享 uniform uFx 门控（dials 取 1、collider 取 0），力场碰撞模式的渲染结果不变。

6.4 质量分级与硬件降级

三档画质的差异只有一个数字：模拟纹理边长 256 / 512 / 1024，对应 65,536 / 262,144 / 1,048,576 粒子。

两条自动降级路径：

- 显存格式降级：若 renderer.extensions.has('EXT_color_buffer_float') 为假，自动改用 HalfFloatType 纹理，避免在移动 GPU / 老核显上初始化失败。
- 帧率降级：每 500ms 统计一次 FPS；若连续 3 秒低于 45，自动下降一档并 Toast 提示。只降一次，不回弹——来回切换档位比稳定低画质更影响观感。

7 六个场景的实现

六个场景共享同一套引擎，差异全部封装在各自的 *Force() 函数与 config 常量中。以下只讲技术要点，不重复产品描述。本章配图均在 1440×900 视口、med 画质（262,144 粒子）下实际运行截取，未做后期修饰。

> 关于配图里的 FPS 读数：截图环境是无 GPU 加速的无头浏览器（软件光栅化），因此 HUD 上显示的个位数帧率只反映该渲染后端，不代表真机性能。本作品不使用任何未实测的性能数据，真机帧率见《实测记录表》。

7.1 星云 nebula

- 形态：球壳上的湍流云 → 致密小球。dial 驱动 targetRadius 从 2.3 降到 0.45，刚度从 2.0 升到 4.8。
- 湍流分层：三层 octave 的噪声分别对应「柱状结构 / 中层飘带 / 细丝」，再叠加一个与主旋向相反的切向分量，制造差速旋转的真实感。
- 厚度控制：uNebulaAxis 三轴缩放让云团呈扁椭球，避免看起来像一颗标准球。



图 7-1　nebula：三层噪声叠加出柱状结构、中层飘带与细丝，边缘保留稀疏挥发感

7.2 星系 galaxy（星锻）

- 播种：指数分布 r = -ln(1-0.985u)/3.2，越靠中心越密；盘厚随半径线性增长 + 中心 bulge。
- 差速旋转：切向初速 v = orbitSpeed · r/(r+0.5)，天然给出内快外慢。
- 星锻（本项目重做）：握拳时 uForge 从 0→1，所有粒子按立方根半径（体积均匀）分配一个球内固定槽位，以高刚度 + 逐渐衰减的旋量锁住，盘面熔铸成一颗致密发光球。
- 着色：热白核 → 金色边；随机粒子迸发十字星芒；加性叠加下整体压暗以保证星芒可辨。



图 7-2　galaxy「星锻」：旋臂向内螺旋熔铸为致密发光体，随机粒子迸发四点星芒；左上角 HUD 显示 STAR FORGE 已拉满

7.3 粒子地球 ocean（整体重写）

- 球面采样：cosθ = 2u−1 等面积采样，避免两极堆积；半径 0.96–1.00 倍 containRadius 的薄壳。
- 自转：初速给东向漂移；力场中再按纬度加权切向驱动（赤道快、两极慢），否则刚体会「像贴图在转」。
- 轮廓保持：当前湍流直接叠加，尚未实现完整的球面切平面投影；需实机核对轮廓稳定性。
- 极光：按 seed 选定固定 22% 粒子，握拳时沿「波浪形极区椭圆」抬升 0.55 高度，形成上下贯通的极光幕；颜色从脚部绿渐变到顶端品红；权重为乘性，uAurora=0 时严格归零。
- 着色：两层 value noise 生成海陆云三分色 + 昼夜明暗。

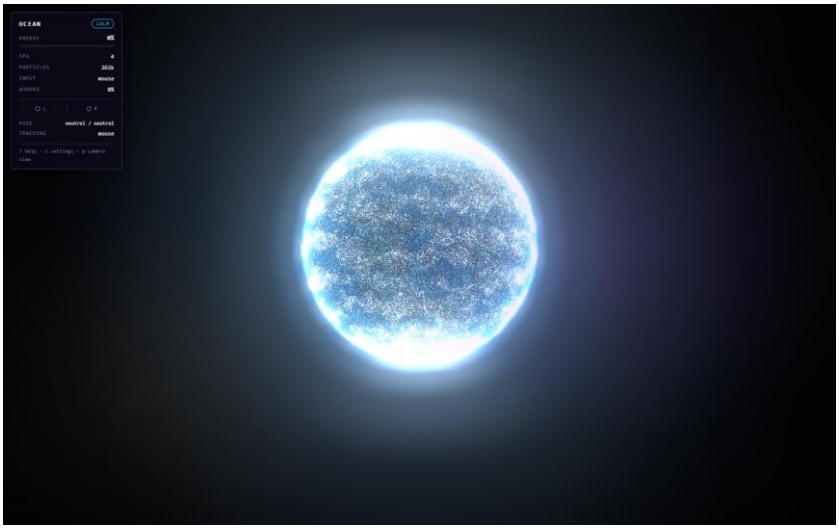


图 7-3　ocean：粒子地球。薄球壳等面积采样保证两极不堆积，当前噪声扰动未完整投影到球面切平面，轮廓表现须实机验证

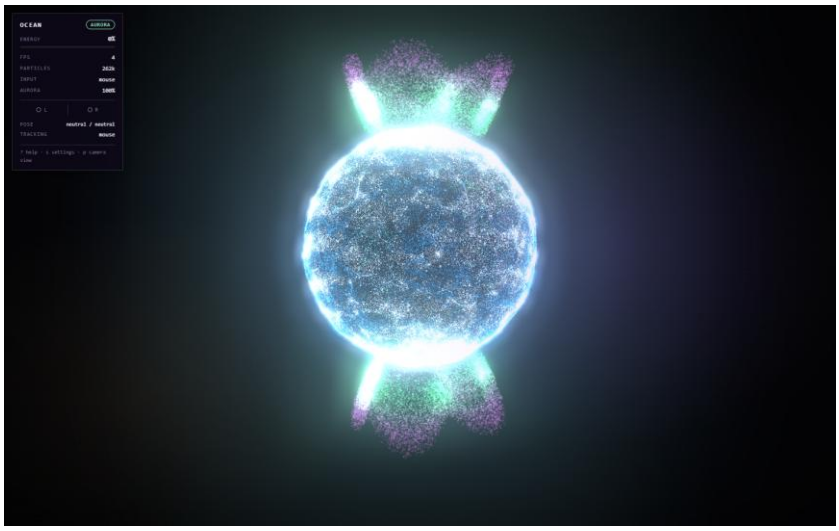


图 7-4　ocean「极光风暴」：22% 粒子抬升为极区极光幕，脚部绿色、顶端品红；HUD 显示 AURORA 已拉满

7.4 土星 saturn

- 双角色粒子：按 particleRoleThreshold = 0.34 分配，约 2/3 为环系、1/3 为行星本体。
- 卡西尼缝：播种时用拒绝采样，落在 [1.72, 1.90] 内的粒子按 50% 概率挤到缝两侧，形成真实密度空档。
- 倾斜坐标系（本项目关键修复）：把位置与速度一并旋转进倾斜环面局部系，在局部系里算切向速度、垂直回正与轨道驱动，再把合力转回世界系。早期只旋转位置不旋转速度，导致倾斜环面长时间运行后整体倾覆。
- 开普勒差速：v = orbitSpeed · r^(−0.5)，指数由 keplerExponent 控制，可现场改成 0 做对照演示。

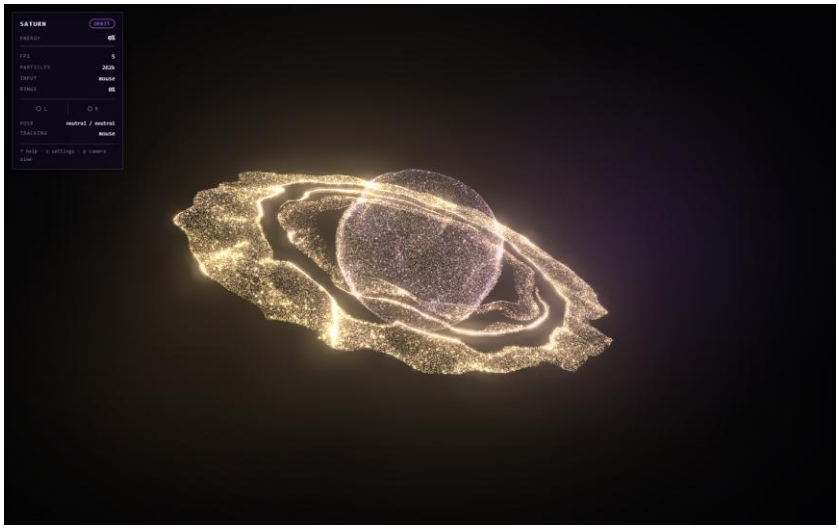


图 7-5　saturn：倾斜环系。环面上的暗带即卡西尼缝，由播种阶段的拒绝采样直接生成

7.5 心跳 heart

- 几何：隐式心形 $(x^2+y^2-1)^3 - x^2y^3 \le 0$ 的面积均匀拒绝采样。相比参数方程径向缩放，隐式解法在心尖与凹口处无接缝。
- 深度剖面：f 在中心为 1、轮廓处为 0，取立方根得到「枕形」厚度，多数粒子落在前后表皮，轮廓从任意角度都可辨认。
- 节律：相位积分 ph += dt·BPM/60，避免 BPM 变化时相位跳变；lub 主峰位于 0.04（宽 16）、dub 回声位于 0.30（幅度 0.55、宽 18），取二者 max。
- 冲击波：每次搏动从中心发出一个向外传播的环形扰动，随 dial 增强。
- 暗物质环：三层不同倾角的轨道环，握拳时环被拉近并加速旋转。

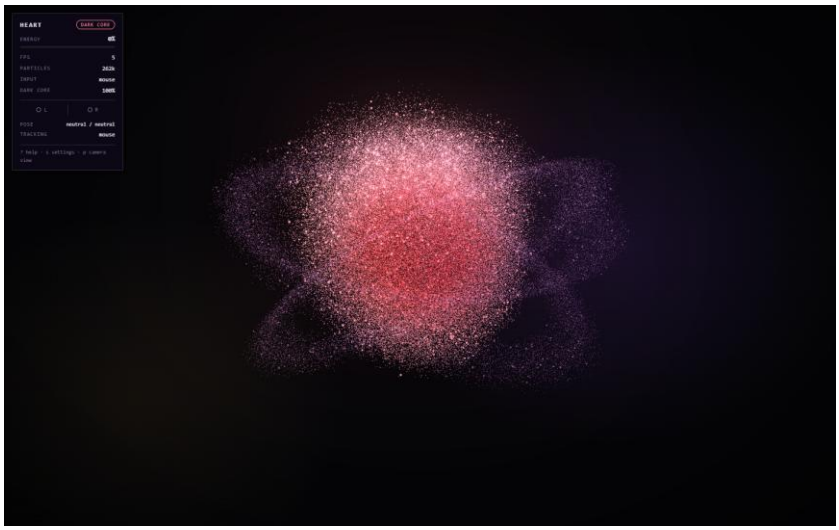


图 7-6　heart：隐式心形采样出的点云，主体落在前后表皮，因此从任意角度都能读出轮廓；图为拨盘拉满时的「暗核」状态，中心被掏空、边缘转亮，HUD 显示 DARK CORE 100%

7.6 离子脉冲 pulse

- 分 6 组、周期 3 秒错峰点火；88% 粒子落在球壳上（快速），其余为慢速余烬。
- 弹道：点火后仅受重力与阻尼，形成抛物线；拨盘到 1 时切换为「离子环」——向心汇聚、压平成环状冲击波、并被召回。
- 过曝治理：召回目标是半径 0.1 量级的小壳层而非一个点，并在着色中压暗近核区域，保证环状结构可见。
- 环面朝向：每组爆发拥有独立且固定的法线倾角，避免所有环侧视叠成一摞。

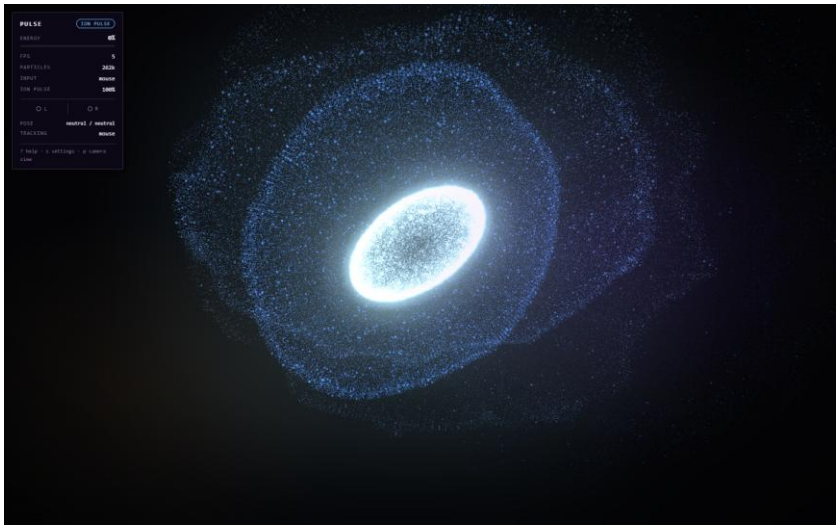


图 7-7　pulse「离子环」：爆发向心汇聚并压成环状冲击波；召回目标为小壳层，因此核心不会过曝成一团白斑

8 AI 与 Skills 协同开发及意图控制

本章按作者提供的使用情况说明工具分工，并以现有代码及复盘材料解释上下文筛选和验收方式。工具名称与参与环节来自作者陈述；回溯复盘并非原始聊天记录。

开发主线使用 Claude Opus 5.5；前端设计使用 Skills 与 ChatGPT 6.0 sol；粒子特效设计也参考 Skills，并由 Opus 5.5 优化；总项目完成后使用 DeepSeek Flash 检查与修缮；最终文稿由 DeepSeek 撰写，再经 ChatGPT 修缮。这是作者提供的实际工作分工，不代表这些工具共同生成了每一处代码。

8.1 工具分工与 Skills 使用边界

Skills 在这里是任务规范与方法参考，并非项目的运行依赖。前端优先选用适合原生 TypeScript/DOM 的视觉层级、间距、文案和状态反馈规则；粒子效果侧将适用的效果设计思路转化为可验证的形态与运动约束，再由 Opus 5.5 协助优化。未核对到具体 Skill 名称与原始调用记录的部分，不虚构 Skill 清单。

层	内容	什么时候给	为什么这样切
L0 规范层	与框架无关的设计约定：间距尺度、层级节奏、状态归属的判断标准、界面文案的语气与长度、错误提示的写法	整个开发周期常驻	决定「长期一致性」，不决定「这一次做得对不对」
L1 契约层	config.ts 的常量段、目标函数的签名与返回值约定	永远给	这是 AI 与人之间的共同语言，不给它就会自创接口
L2 结构层	同类函数的既有实现（如 galaxyForce）、StateMachine 的 switch 结构	改同类逻辑时给	示范风格，避免它引入第二套写法
L3 实现层	完整文件、UI 文件、未涉及的着色器	默认不给	上下文越大，AI 越倾向「顺手重构」你没让它碰的地方

上下文按 L0 规范、L1 接口契约、L2 相邻实现、L3 完整文件分层（见下表）；只提供当前任务需要的层级，以减少无关改动。表中是复盘总结的方法，不是逐轮提示词的原始导出。

Skills 筛选原则

参考规范需与项目技术栈匹配：UI 是原生 TypeScript 与 DOM，故保留框架无关的设计原则，不把 React hooks 或组件规则当作实现要求；粒子效果须回到 GLSL 力场、播种、着色与验收条件，而不是仅凭效果描述直接合并。

8.2 约束式 Prompt 与代码验收

模式	模板	解决什么问题
边界先行	「只改 X 文件，不许动 Y/Z 的任何现有表现」	防止顺手重构，把改动面锁死

模式	模板	解决什么问题
先方案后代码	「先说方案，不要写代码。重点回答 a、b 两个问题」	错误方向在成本最低的时候被拦下
可验证判据	「线速度按 v ∝ r^(-0.5) 变化，角速度按 ω ∝ r^(-1.5) 变化；具体比值须给定半径后计算」	把「要真实」这类形容词换成可测条件
现象+隔离结论	「关掉 X 就正常，说明问题在 X」	AI 对具体证据的回答质量远高于对「为什么不稳」
要求自查清单	「列出你改了哪几行、为什么」	既是人的 review 清单，也逼它检查越界

8.3 典型案例：倾斜环面的坐标系失稳

以下是根据代码与阶段快照回溯的技术案例，详见 prompts/01；该文件不是原始 AI 对话，具体对话轮次与时长仍需原始记录核验。

「症状：土星环倾斜 0.42 rad 时，长时间运行后整体倾覆。」

错误的做法是直接把症状丢给 AI。正确做法是先做三次变量隔离：

实验	操作	结果	结论
1	tilt 设为 0	稳定	问题与倾斜强相关
2	湍流设为 0	仍然倾覆（更慢）	排除湍流为主因
3	轨道驱动设为 0	完全稳定	锁定在切向轨道驱动

根据回溯复盘，归零对照将问题定位到轨道驱动：位置在局部坐标系、速度仍在世界坐标系，造成倾斜环面的速度分量误判。代码中的修法是位置与速度一起转入局部系，算完再转回。原始问答内容尚待核对。

> 这一案例强调先定位变量、再提出有边界的修改要求；运行表现与持续时长须以实测记录为准。

8.4 三种失败模式与对策

失败模式	表现	根因	对策
只做一半的变换	旋转了位置，忘了速度	坐标系变换类需求，AI 容易漏掉派生量	Prompt 中点名「位置和速度都要变换」
形容词驱动	「要成片」「要自然」→ 得到散点云/畸形结果	不可验证的目标无法收敛	翻译成数学条件：相邻粒子落点相关性、连续参数而非随机数
魔术数字内联	常数散落在着色器里	AI 倾向局部最优化，忽略项目约定	硬约束「所有系数走 uniform」，并在 review 时 grep 数字

8.5 人的判断边界

有几类决定，我始终自己做，不让 AI 代劳：

8. 产品定位与取舍——例如「手势优先、鼠标提供基础操作兜底」，这是价值判断，不是技术选择。
9. 验收标准——在写第一行 code 之前就把「什么算做完」写成可验证的句子。
10. 上下文投喂范围——哪些规范该给 AI、哪些必须剔除，由人判断。给错了比不给更糟。
11. 性能预算——粒子数上限、每帧 pass 数、着色器复杂度上限，由人拍板。
12. 架构边界——模块职责表，以及「新增场景只许改四处」这条纪律。
13. 对外披露口径——开源来源如何陈述、原创贡献如何界定，涉及诚信，必须由人负责。

9 工程质量

9.1 类型安全、构建与性能设计

全量 TypeScript 严格模式，npm run build 先跑 tsc 再跑 vite build，类型不过就不产出产物；当前 tsc --noEmit 零错误。GLSL 通过 Vite 的 ?raw 内联，构建期完成字符串拼接，运行时不需要再请求着色器文件；common.glsl 抽为公共库，三个着色器在编译期以字符串前缀注入，既复用噪声函数又不依赖 WebGL 的 #include 扩展。

性能上有四条纪律：模拟与渲染分离（GPGPU pass 只输出纹理，渲染 pass 只采样纹理，互不阻塞）；手部检测 30Hz 抽帧，与 60Hz 渲染解耦；页面隐藏时暂停检测，后台不烧 GPU；pixelRatio 上限 2，防止 4K 屏像素数翻四倍导致掉帧。

9.2 资源生命周期

资源	创建时机	释放时机	释放动作
GPGPU 纹理与材质	构建粒子系统时	切换画质 / 重置 / 退出	gpu.dispose() + geometry/material.dispose()
摄像头流	进入摄像头模式	Esc / 退出到启动页	getTracks().forEach(stop) + video.remove()
MediaPipe 模型	进入摄像头模式	同上	landmarker.close()
检测定时器	同上	同上	clearTimeout + disposed 标志双保险

切换画质时先 remove 旧的 Points、dispose，再 new 新的，避免显存泄漏。六场景连续切换与反复切换画质的稳定性仍应按《实测记录表》记录设备、步骤与结果。

9.3 缺陷复盘

#	缺陷	根因	修复
1	土星环长时间运行倾覆	位置进局部系、速度未进	位置与速度一并变换
2	土星内环比外环慢	切向驱动用了常数角速度	改为 v ∝ r^(−0.5)
3	土星本体高度失控	垂直只有弹性、无阻尼	显式垂直阻尼 + 压缩纹理扰动
4	地球膨胀、边缘发毛	湍流带径向分量	当前代码尚未完整实现噪声切平面投影，需进一步修复与验证
5	极光成散点不成片	幕布脚用随机方位角	改连续参数 + 低频谱波
6	极光关闭后残留薄雾	抬升用 mix 有偏置	改乘性权重
7	心跳换 BPM 时卡顿	相位 = t × BPM	相位积分 ph += dt·BPM/60
8	离子环核心过曝	召回目标是一个点	召回到小壳层 + 近核压暗
9	离子环全部侧视	环法线固定	每组随机固定倾角
10	换画质档亮度漂移	加性混合亮度 ∝ 粒子数	uDensityNorm 归一

10 可复现性论证

10.1 环境基线

项	版本/要求	说明
Node.js	≥ 18（推荐 20 LTS / 22）	仅构建期需要
包管理器	npm ≥ 9，使用 package-lock.json	建议 npm ci 保证锁定版本
浏览器	Chrome/Edge 113+ 为首选	必须支持 WebGL2
操作系统	Windows 10/11、macOS 12+、主流 Linux	纯前端，无平台特定代码
网络	首次运行需联网	下载 npm 依赖与 MediaPipe CDN 资源

本项目不依赖任何环境变量与密钥。完整说明与扩展模板见 README 的「配置与环境变量」一节。

10.2 复现路径

```
# 方式 A（Windows 一键）
双击 start.bat

# 方式 B（命令行）
npm ci
npm run dev          # 打开终端打印的 localhost 地址

# 方式 C（静态产物）
npm run build
npm run preview
```

10.3 八项验证清单

#	操作	预期	失败时的排查方向
1	打开页面	出现启动页，无白屏/无控制台报错	控制台是否报 WebGL2 unavailable
2	鼠标模式按住左键	星云收束成球，松开回弹	输入层是否产出 GestureFrame
3	按 G	场景循环切换，HUD 同步	SCENES 数组与 switch 是否一致
4	右键拖拽	粒子场跟随旋转，松手带惯性	轨迹球角速度积分
5	按 S	设置面板打开，可调画质/泛光/粒子尺寸	Overlay 回调是否接上
6	摄像头模式举手	出现手部标记，开合驱动形态	权限、模型加载、光照
7	按 H	HUD 显隐	键盘监听
8	按 Esc	回到启动页，摄像头指示灯熄灭	dispose 链是否执行

10.4 证据链

复现证据包括源代码与依赖锁文件、README、技术文档及实测记录；三篇 prompts/ 是事后复盘，不能替代原始 AI 对话。

证据	位置	能证明什么
Git 提交历史	仓库根目录	现有提交反映阶段代码状态；早期快照于 2026-10-28 回溯整理，不能证明原始开发时间线
Prompt 历史快照	prompts/（3 篇事后复盘；原始工具对话尚未取得）	仅用于说明技术复盘；实际 AI 协同轮次须以原始工具记录证明
代码内注释	velocity.frag.glsl 等	可供核查实现意图，不能代替同期实验或原始对话证据
验证清单	README + 本文 10.3	提供复现步骤与检查清单；实际完成时间和功能需独立验证

11 开源合规与原创性

11.1 上游与许可

项	内容
上游项目	gesture-particles
作者	William Larskog
许可证	MIT（LICENSE 原样保留，版权与许可声明未删改）
灵感出处	David Katz 的 MediaPipe + Three.js 粒子模拟器
第三方 GLSL	Ashima Arts webgl-noise 单纯形噪声（公有领域）

11.2 本项目的实质性贡献

类别	内容	可验证点
新增场景 ×3	saturn / heart / pulse	velocity.frag 新增 6 个函数；config 新增 42 个常量
重写场景 ×1	ocean 由平面波浪改为粒子地球 + 极光	waveHeight() 被删除，新增 auroraForce()
机制重做 ×1	galaxy 黑洞坍缩改为星锻熔铸	新增 forgeForce()，核心保持明亮
架构重构	抽取 common.glsl；均匀参数共享；跨档亮度归一	common.glsl 为新增文件
产品化	全站中文化、启动页、HUD、Toast、参数面板	index.html 标题与 meta 重写

类别	内容	可验证点
工程化	严格类型检查、CI 部署	tsc --noEmit 零错误

11.3 为什么主动披露

赛题禁止的是「换皮参赛」，不是「二次开发」。规则原文同时写着「可以参考或二次开发开源项目，但须核实许可证，遵守署名、版权声明及其他许可条件」。本项目的选择是：

- 在 README 顶部与 ORIGINALITY.md 中主动说明上游、作者、许可证；
- 把「我改了什么」写成可核对的表格，而不是含糊的「进行了优化」；
- 保留 LICENSE 原文与版权声明，不做任何修改；
- 保留本地 Git 历史并说明来源：早期阶段提交是 2026-10-28 依据目录快照回溯整理，不能作为开发当日原始提交证明；详见 ORIGINALITY.md。

这样做的结果不是「暴露短板」，而是让「增量」变成可验证的事实。

12 局限与后续规划

12.1 已知局限与演进路线

项目	现状	下一步
单目深度估计	手的前后移动分辨率低；深度仅做辅助，主驱动为平面内的开合与位移	评估双目方案的可行性
强光/逆光	教室靠窗时可能丢手；丢手自动落回鼠标，不中断演示	增加识别质量提示
移动端	未做触控适配，明确以桌面浏览器为目标环境	P1：平板触控适配
录制回放	无法把一次演示存成课件	P0：演示录制与一键回放（含手势轨迹）
教学模式	场景内暂无知识点卡片	P0：内嵌知识点卡片与参数标注，把「炫技」转成「教具」
自定义场景	场景为内建实现	P2：开放插件接口，允许用户提供 GLSL 片段

A 附录：关键配置参数

场景	参数	值	含义
通用	damping / maxSpeed	0.965 / 4.2	每帧速度衰减与速度上限
通用	detectHz / maxDelta	30 / 0.033s	手部检测频率 / 单帧时间步上限
通用	fpsDowngradeBelow	45	自动降档阈值
oneEuro	minCutoff / beta / dCutoff	1.0 / 0.02 / 1.0	滤波参数
nebula	cloudRadius / ballRadiusTight	2.3 / 0.45	云团与致密球半径
galaxy	discRadius / discRadiusCollapsed	30.6 / 2.9	盘半径与其坍缩值
galaxy	forgeRadius	0.85	星锻实心球半径
ocean	containRadius / auroraFraction	1.28 / 0.22	球半径 / 参与极光的粒子比例
ocean	auroraHeight	0.55	极光抬升高度
saturn	innerRing / outerRing	1.24 / 2.72	环内外半径
saturn	cassiniGapStart / End	1.72 / 1.90	卡西尼缝区间
saturn	tilt / keplerExponent	0.42 / 0.5	环面倾角 / 开普勒指数
heart	bpmCalm / bpmIntense	62 / 128	心率区间
pulse	period / groups	3.0 / 6	脉冲周期 / 分组数

B 附录：快捷键与交互索引

输入	动作	效果
键盘	G / S / H	切换场景 / 设置面板 / HUD 显隐

输入	动作	效果
键盘	P / R / C	摄像头预览 / 重置粒子 / 切换 dials·collider
键盘	? 或 / · Esc	帮助浮层 · 断开摄像头返回启动页
左手	开合程度	调节盘（场景形态轴）
右手	位移	轨迹球旋转（带惯性）
双手	张开程度 / 距离变化速度	扩散冲击波 / 引力强度
手部	移动速度	能量（提亮与湍流）
鼠标	按住左键 / 右键拖拽	等价于握拳 / 等价于右手位移（旋转）
鼠标	滚轮 / 移动速度	锁存「按住左键」/ 等价于手速（能量）